Priority Engine

Pull Request Traffic Controller

Technical Explanation, Scoring Logic & Interview Preparation

The Priority Engine is a scoring system that examines a pull request and asks: **“How dangerous or urgent is this PR for the team's workflow?”** Each factor contributes points. More points mean higher priority.

Example: Age +40, Pending Reviews +20, Conflict +50, Failing CI +40, Blocking Others +100, Bug +50 = **300 points** → **Critical**.

1. Age of the PR

How long the PR has been open. Longer-open PRs may indicate staleness, review delays, unresolved dependencies, or other workflow friction.

Condition	Points
0–2 days	+10
3–5 days	+20
6–10 days	+40
10+ days	+60

Example: PR #101 is 8 days old → 6–10 days → **+40**.

2. Pending Reviews

How many requested reviewers are still expected to review the PR. More pending reviews can mean more coordination and waiting.

Condition	Points
0	+0
1	+10
2	+20
3+	+30

Example: Three reviewers are requested; one approved and two are pending → **+20**.

3. Merge Conflict

Whether GitHub reports that the PR cannot currently be merged cleanly because of a conflict that needs resolution.

Condition	Points
No conflict	+0
Conflict	+50

Example: PR #103 has mergeable = false → **+50**.

4. Failing CI

CI (Continuous Integration) includes automated builds, tests, linting and similar checks. A failing check signals that the PR needs attention.

Condition	Points
CI passing	+0
CI failing	+40

Example: Build and lint pass, but unit tests fail → **+40**.

5. Is Blocking

Whether the PR is blocking other PRs through dependency relationships. A blocking PR can delay downstream work, so it receives a strong weight.

Condition	Points
Blocking 0 PRs	+0
Blocking other PRs	+100

Example: #102 depends on #101 and #103 depends on #102. Delaying #101 can delay work behind it → potentially **+100**.

6. Urgent Label

A GitHub “urgent” label is an explicit human priority signal indicating that the PR needs quick attention.

Condition	Points
No urgent label	+0
urgent	+80

Example: Age +10 and urgent +80 gives a subtotal of **90**.

7. Bug Label

A “bug” label indicates that the PR fixes a defect. Bug fixes may deserve more attention than ordinary feature work.

Condition	Points
No bug label	+0
bug	+50

Example: A PR fixing an image-upload failure is labelled bug → **+50**.

8. Security Label

A “security” label indicates a potentially high-risk issue involving areas such as authentication, authorization, data or infrastructure.

Condition	Points
No security label	+0
security	+100

Example: A PR fixing an authentication vulnerability has security → **+100**.

Putting Everything Together

Suppose GitHub provides this information for PR #221:

Signal	Value	Score
Age	8 days	+40
Pending reviews	2	+20
Merge conflict	Yes	+50
CI	Failing	+40
Blocking	2 PRs	+100
Label	bug	+50
Total		**300**

Using the example buckets: **0–40 Low, 41–80 Medium, 81–120 High, 120+ Critical**. Therefore, 300 points → **Critical**.

How the Data Flows

Stage	Purpose
GitHub API	Provides PR metadata, reviewers, labels, timestamps, checks and mergeability.
PR Data	Raw information becomes inputs to the scoring system.
Signals	Age, reviews, conflict, CI, blocking and labels are evaluated.
Dependency Data	Relationships reveal workflow impact and blocking.
Priority Engine	Configured weights are added together.
Result	The system returns a score and a priority level.

Interview Explanation

Short answer: “The Priority Engine is a rule-based scoring system. It takes signals such as PR age, pending reviews, merge conflicts, failing CI, dependency-based blocking, and GitHub labels. Each signal has a configured weight. The engine sums the weights and maps the final score to Low, Medium, High or Critical. This lets a team prioritize work based on workflow impact rather than simply looking at PR creation order.”

Important distinction: The current engine is **rule-based prioritization**, not machine-learning prediction. To claim predictive ML, you would need historical PR data and a trained model that predicts an outcome such as review delay or time-to-merge.

Key Interview Points

• The engine does not simply inspect the PR title.

• Most signals originate from GitHub PR metadata, review information and CI/check information.

• Dependency relationships provide workflow-impact information, especially for identifying blocking PRs.

• The scoring is explainable because each point comes from a known rule.

• Weights can be tuned according to a team's risk and workflow priorities.

• ML can be added later for learned prediction, while the current deterministic engine remains transparent.